LABSHEET 6 : FULL STACK DEVELOPMENT

NAME : DHATRI KASHYAP

COURSE : BTECH CSE 'B'

ROLLNO : 18

Q1. Server Setup & Building the REST API

- **Create a folder api-lab with server/ and client/ subfolders. Inside server/, initialize npm, install express, cors, and dotenv, and set up a .env file with PORT=5000 (read via process.env.PORT with a 5000 fallback). Add a "dev" script using nodemon.**
- **Using an in-memory array of at least 5 “task” objects (id, title, completed), build all 5 CRUD routes: GET /api/tasks, GET /api/tasks/:id, POST /api/tasks, PUT /api/tasks/:id, and DELETE /api/tasks/:id — with a proper 404 JSON error when an id doesn't exist.**
- **Write a custom logger middleware (method, URL, timestamp) applied globally, and an auth-check middleware requiring a header x-api-key (stored in .env) applied only to the POST, PUT, and DELETE routes.**

ANS : (a) Create the project structure:

mkdir api-lab cd api-

lab mkdir server

client **Initialize the**

server: cd server

npm

init -y

Install required dependencies:

npm install express cors dotenv

npm install --save-dev nodemon

Create .env file (in server/):

PORT=5000

API_KEY=your_secret_key_here # Replace with a strong key (e.g., generated via openssl rand -hex 16) **Update**

package.json (add the dev script):

{

"name": "api-lab-server",

"version": "1.0.0",

"main": "server.js",

"scripts": {

```
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5", "express":
    "^4.19.2"
  },
  "devDependencies": {
    "nodemon": "^3.1.4"
  }
}
```

(b) In-Memory Tasks and CRUD Routes Create

server.js in server/ with this code:

```
require('dotenv').config(); const express =
require('express');          const       cors       =
require('cors'); const app = express(); // Middleware
          app.use(express.json());
app.use(cors()); // Enabled for development
// In-memory task array (min 5 objects) let
tasks = [
  { id: 1, title: 'Learn Node.js', completed: false },
  { id: 2, title: 'Build REST API', completed: true },
  { id: 3, title: 'Test with Postman', completed: false },
  { id: 4, title: 'Create JS client', completed: false },
  { id: 5, title: 'Develop React frontend', completed: false }
];
// Custom logger middleware (global)
app.use((req, res, next) => { const timestamp = new
  Date().toISOString();          console.log(`[${timestamp}]
```

```
 ${req.method} ${req.url}`); next();
});
// Auth-check middleware (for POST, PUT, DELETE only) const
authenticate = (req, res, next) => { const apiKey =
req.headers['x-api-key']; if (apiKey && apiKey ===
process.env.API_KEY) {
   next();
  } else { res.status(401).json({ error: 'Unauthorized: Missing or invalid API key'
  }); }
};
// CRUD Routes
// GET /api/tasks - Read all
app.get('/api/tasks', (req, res) => { res.json(tasks);
});
// GET /api/tasks/:id - Read one app.get('/api/tasks/:id', (req, res) => { const task =
tasks.find(t => t.id === parseInt(req.params.id)); if (!task) { return res.status(404).json({
error: `Task with id ${req.params.id} not found` });
  } res.json(task);
});
// POST /api/tasks - Create (protected) app.post('/api/tasks',
authenticate, (req, res) => { const
{ title, completed } = req.body;   if (!title) { return
 res.status(400).json({ error: 'Title is required' });
  } const newTask = { id: tasks.length ?
  Math.max(...tasks.map(t => t.id)) + 1 : 1,
   title,
   completed: completed || false
  };
  tasks.push(newTask); res.status(201).json(newTask);
});
```

```
// PUT /api/tasks/:id - Update (protected) app.put('/api/tasks/:id', authenticate, (req, res)
=> { const task = tasks.find(t => t.id === parseInt(req.params.id)); if (!task) { return
res.status(404).json({ error: `Task with id ${req.params.id} not found` });
  } const { title, completed } = req.body; if (title !== undefined)
  task.title = title; if (completed !== undefined) task.completed
  = completed; res.json(task);
});
// DELETE /api/tasks/:id - Delete (protected) app.delete('/api/tasks/:id', authenticate,
(req, res) => { const index = tasks.findIndex(t => t.id === parseInt(req.params.id));
if (index === -1) { return res.status(404).json({ error:
`Task with id ${req.params.id} not found` });
  } const deletedTask = tasks.splice(index,
  1)[0]; res.json(deletedTask);
});
// Start server const PORT = process.env.PORT ||
5000;          app.listen(PORT,        ()        =>        {
console.log(`Server running on port ${PORT}`);
});
```

(c) Middleware Implementation Details

Logger Middleware (lines 12-18 in server.js):

Applied globally via app.use()

Logs: [timestamp] METHOD URL (e.g., [2024-06-15T10:30:00.000Z] GET /api/tasks)

Uses new Date().toISOString() for consistent timestamps

Auth-Check Middleware (lines 21-30):

Only applied to POST, PUT, and DELETE routes (see route definitions)

Checks for x-api-key header matching process.env.API_KEY

Returns 401 with JSON error if missing/invalid

Q2. Securing, Testing & Error-Handling the API

LABSHEET 6 : FULL STACK DEVELOPMENT

- **Enable CORS for development. Temporarily disable it, note the exact browser error when calling the API from a client, then re-enable it and confirm the fix.**

- **Create a Postman (or Thunder Client) collection named "API Lab" covering all 5 endpoints, including the x-api-key header where required. Confirm GET works without the key but POST returns 401 without it. Screenshot one successful POST request (body + response).**

- **Add a 404 handler (after all routes) for unmatched URLs, and a centralized error-handling middleware (4 parameters) that returns a consistent JSON error shape for any thrown error.**

ANS : (a) CORS Testing Procedure

Temporarily disable CORS (in server/server.js):

// Comment out or remove this line temporarily:

// app.use(cors());

Test from a client (use the plain JS client from Q3a or create a simple test file):

Create test-cors.html in client/:

```
<!DOCTYPE html>
<html>
<body>
  <script> fetch('http://localhost:5000/api/tasks')
     .then(res => res.json())
     .then(data => console.log(data))
     .catch(err => console.error('Fetch error:', err));
  </script>
</body>  </html>
```

Open this file directly in a browser (file:// protocol) or serve it via a simple server (e.g., npx serve client on port 3000).

Exact browser error observed (in Console tab):

Access to fetch at 'http://localhost:5000/api/tasks' from origin 'null' has been blocked by CORS policy: No 'AccessControl-Allow-Origin' header is present on the requested resource. If an opaque response serves your needs, set the request's mode to 'no-cors' to fetch the resource with CORS disabled.

(Note: Origin may appear as null for file:// or http://localhost:3000 if served locally) Re-enable

CORS (uncomment app.use(cors()); in server.js).

Confirm fix: Refresh the client page → Data loads successfully without console errors.

(b) Postman Collection Setup Steps to

create "API Lab" collection:

In Postman:

Click New → Collection → Name: API Lab

Add these requests (all under http://localhost:5000):

GET /api/tasks

Method: GET

URL: /api/tasks

Headers: None (GET doesn't require auth)

Test: Send → Should return 200 with task array

GET /api/tasks/:id

Method: GET

URL: /api/tasks/1

Headers: None

Test: Send → Returns task with id=1 (200); change to 999 → 404 error

POST /api/tasks

Method: POST

URL: /api/tasks

Headers:

Content-Type: application/json x-api-key:

[your_key_from_.env]

Body → raw → JSON:

```
{
  "title": "New task from Postman",
  "completed": false
}
```

Critical test:

Without x-api-key header: Send → Returns 401 { "error": "Unauthorized: Missing or invalid API key" }

With valid x-api-key: Send → Returns 201 with new task object (including auto-generated id)

PUT /api/tasks/:id

Method: PUT

URL: /api/tasks/1

Headers: Same as POST (Content-Type, x-api-key)

Body:

```
{
  "title": "Updated title",
  "completed": true
  }
```

Test: Send → Returns updated task (200); try id=999 → 404

DELETE /api/tasks/:id

Method: DELETE

URL: /api/tasks/1

Headers: Same as POST

Test: Send → Returns deleted task object (200); try id=999 → 404

Screenshot description for successful POST

Headers tab: Shows Content-Type: application/json and x-api-key: [your_key]

Body tab: Raw JSON with "title": "Lab task", "completed": false

Response tab: Status:

201 Created

Body:

```
{
  "id": 6,
  "title": "Lab task",
  "completed": false
}
```

(c) Error Handling Implementation

Add these to server/server.js after all route definitions but before app.listen():

LABSHEET 6 : FULL STACK DEVELOPMENT

```
// 404 Handler for unmatched URLs (must come after all API routes) app.use((req,
res) => { res.status(404).json({ error: `Route
${req.originalUrl} not found` });
});
// Centralized error-handling middleware (4 parameters) app.use((err, req,
res, next) => { console.error(`[ERROR] ${err.stack}`); // Log full error
for debugging
  // Default to 500 if no status set const status
  = err.status || 500; // Consistent JSON error
  shape res.status(status).json({ error:
  err.message || 'Internal Server Error'
  });  });
// Example usage in routes (throw errors to trigger handler):
// In POST route validation:
if (!title) { const error = new Error('Title is
  required'); error.status = 400; // Set custom
  status throw error;
}
```

Q3. Consuming the API — Plain JavaScript Client

Inside client/, create a simple index.html and script.js (no framework) that calls GET /api/tasks using fetch() and renders the task titles as a list.

Add loading and error handling in plain JS: show "Loading..." while the request is in progress, and a clear error message if the fetch fails.

Test the failure path by stopping the server while the page is open, and confirm the error message appears correctly instead of the page breaking.

ANS : Consuming the API — Plain JavaScript Client

(a) & (b) Client Implementation: index.html and script.js

Create these files inside the client/ folder (which already exists from your Q1 setup): **client/index.html:**

```
<!DOCTYPE html>
<html lang="en">
<head>
```

LABSHEET 6 : FULL STACK DEVELOPMENT

```
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Task List - Plain JS Client</title>
    <style> body { font-family: Arial, sans-serif; margin:
       2rem; }
       #loading { color: #666; font-style: italic; }
       #error     {          color:    #d32f2f;
          background-color: #ffebee;
          padding:          1rem;
                  borderradius: 4px; margin:
          1rem
          0; display: none;
       }
       #task-list { list-style:
          none;  padding:  0;
          max-width:
          500px;
       }
       #task-list li { padding: 0.75rem; border-bottom:
          1px solid #eee;
       }
       #task-list li:last-child { border-bottom: none; }
    </style>
</head>
<body>
    <h1>My Task Manager</h1>

    <div id="loading">Loading tasks...</div>
```

```
    <div id="error"></div>

    <ul id="task-list"></ul>

    <script src="script.js"></script>
</body> </html>
```

client/script.js:

```
document.addEv
entListener('DO
MContentLoade
d', () => {

    // Cache DOM elements const loadingEl =
    document.getElementById('loading'); const errorEl =
    document.getElementById('error'); const listEl =
    document.getElementById('task-list');

    // Configuration (will be moved to config in Q5) const
    API_URL = 'http://localhost:5000/api/tasks';

    // State management functions const
    showLoading = () => {
    loadingEl.style.display = 'block';
    errorEl.style.display = 'none';
    listEl.style.display = 'none';
    errorEl.textContent = '';
    listEl.innerHTML = '';
    };
```

```
const showError = (message) => {
    loadingEl.style.display = 'none';
    errorEl.style.display = 'block'; listEl.style.display
    = 'none'; errorEl.textContent
    = `Error: ${message}`;
};

const showTasks = (tasks) => {
    loadingEl.style.display = 'none';
    errorEl.style.display = 'none';
    listEl.style.display = 'block'; listEl.innerHTML
    = ''; // Clear previous content tasks.forEach(task
    => { const li = document.createElement('li');
    li.textContent = task.title;
    listEl.appendChild(li);
    });
};

// Fetch and display tasks const fetchTasks
= async () => { showLoading();
    try {
        const response = await fetch(API_URL);

        // Handle HTTP error statuses (4xx, 5xx) if (!response.ok) { throw new
        Error(`Server returned ${response.status}: ${response.statusText}`);
        }

        const tasks = await response.json(); showTasks(tasks);
    } catch (error) {
```

```
        // Handle network errors and thrown errors above showError(error.message);
      }
    };


  // Initial load fetchTasks();
});
```

How This Satisfies Q3 Requirements

(a) GET Request & Rendering:

Uses fetch(API_URL) in fetchTasks() to call GET /api/tasks

On success (response.ok), parses JSON and passes task array to showTasks() showTasks() creates

 elements for each task's title and appends to <ul id="task-list">

Renders as a clean list with basic styling (no frameworks used)

(b) Loading & Error Handling:

Loading State:

Shows "Loading tasks..." in #loading div immediately on DOM load

Hides error/list elements during request

Error Handling:

Catches network errors (e.g., server down) via fetch() rejection

Catches HTTP errors (4xx/5xx) by checking !response.ok and throwing custom error Displays

user-friendly message in #error div (red background) with Error: [details] Never exposes raw

stack traces or breaks UI (all errors caught and displayed cleanly)

(c) Testing Failure Path: Steps to verify:

Start your Express server:

cd server npm run

dev

Open client/index.html in your browser (for best results, serve via static server to avoid file:// quirks:

npx serve client -l 3000 # Then visit http://localhost:3000

Verify success path:

See "Loading tasks..." briefly

Then see task list rendered (5+ items from your in-memory array)

LABSHEET 6 : FULL STACK DEVELOPMENT

Test failure path:

Leave the page open in browser

In server terminal, press Ctrl+C to stop the Express server

Refresh the browser page (or navigate away/back)

Observe:

"Loading tasks..." appears immediately

Quickly replaced by error message

like: Error: Failed to fetch (network

error) or

Error: Server returned 503: Service Unavailable (if server partially responds)

Confirm:

Page does not break (no blank screen, no console errors in red)

Error is clearly visible in styled red box

No JavaScript exceptions thrown (all handled in catch block)

Q4. Consuming the API — React Client

- **Create a React app (Vite recommended) inside client-react/. Build a TaskList component that fetches /api/tasks inside useEffect and renders it using the full 3-state pattern (loading, error, data via useState).**

- **Rewrite the same fetch call using axios instead, and note in a short comment how the code differs (JSON parsing, error handling).**

- **Build a form that POSTs a new task (with the Content-Type and x-api-key headers, and JSON.stringify(body)), updates the UI immediately without a page reload, and add a "Delete" button per task that calls DELETE and removes it from the UI.**

ANS : (a)React App Setup & TaskList Component (3-State Pattern)

Step 1: Create Vite React app

Run these commands in api-lab/ (parent folder):

npm create vite@latest client-react ----- template react

cd client-react npm install npm install axios # For part

(b)

Step 2: Configure environment variables Create client-react/.env:

VITE_API_URL=http://localhost:5000/api

VITE_API_KEY=your_secret_key_here # Copy from server/.env

Important: Add .env to client-react/.gitignore (Vite does this by default).

Step 3: Implement TaskList component Replace

client-react/src/App.jsx with this code: import {

useState, useEffect } from 'react'; import axios

from 'axios';

// Create axios instance with base URL (avoids repetition) const

api = axios.create({ baseURL:

import.meta.env.VITE_API_URL,

});

// TaskList component implementing full 3-state pattern export default

function TaskList() { const [tasks, setTasks] = useState([]); const [loading,

setLoading] = useState(true); const [error, setError] = useState(null); //

Fetch tasks on mount (useEffect with empty deps) useEffect(() => { const

fetchTasks = async () => { setLoading(true); setError(null); try { const

response = await api.get('/tasks'); // No auth needed for GET

setTasks(response.data);

} catch (err) {

// Handle network errors & HTTP errors (4xx/5xx)

setError(err.response?.data?.error || err.message ||

'Failed to load tasks'

);

} finally { setLoading(false);

} };

fetchTasks();

}, []); // Empty deps = run once on mount // Handler

for creating new task (optimistic update) const

handleCreateTask = async (title) => { if

(!title.trim()) return;

// Optimistic UI update: add task immediately with temporary

LABSHEET 6 : FULL STACK DEVELOPMENT

```
ID const optimisticTask = { id: Date.now(), // Temporary ID
(negative to avoid conflicts)  title: title.trim(),
  completed: false, isOptimistic: true, // Flag to
  track optimistic tasks
};        setTasks(prev        =>        [...prev,
optimisticTask]); try { const response = await
api.post('/tasks',
    { title: title.trim(), completed: false },
    { headers:
      {
        'Content-Type': 'application/json',
        'x-api-key': import.meta.env.VITE_API_KEY
      }
    }
  );
  // Replace optimistic task with real task from server
  setTasks(prev        =>        prev.map(task        =>
  task.isOptimistic && task.title === title.trim()
        ? { ...response.data, isOptimistic: false }
        : task
    )
  );
} catch (err) {
  // Rollback optimistic update on error setTasks(prev => prev.filter(task
      =>        !task.isOptimistic));       setError( err.response?.data?.error ||
  err.message ||
    'Failed to create task'
  );
}
```

```
};
// Handler for deleting task (optimistic update) const
handleDeleteTask = async (id) => { // Optimistic UI update: remove
task immediately setTasks(prev => prev.filter(task => task.id !==
id)); try { await api.delete(`/tasks/${id}`, { headers:
{ 'x-api-key': import.meta.env.VITE_API_KEY }
    });
  } catch (err) { // Rollback on error setTasks(prev =>
    [...prev, tasks.find(t => t.id === id)]); setError(
    err.response?.data?.error || err.message ||
      'Failed to delete task'
    );
  }
};
// Render loading state  if (loading) return <div className="text-center py-
8">Loading tasks...</div>;
// Render error state if
(error) return (
  <div className="p-4 bg-red-50 border border-red-200 text-red-800 rounded">
    <strong>Error:</strong> {error}
  </div>
);
// Render data state (task list) return
(
  <div className="max-w-2xl mx-auto p-4">
    <h1 className="text-2xl font-bold mb-6">Task Manager</h1>
    {/* Task creation form */}
    <form onSubmit={(e)
      => {
      e.preventDefault();      const      title      =
```

```
    e.target.elements.title.value;
    e.target.elements.title.value = ''; // Clear input handleCreateTask(title);
  }}    className="mb-6    flex
  gap-3"
>
  <input type="text" name="title" placeholder="Enter new task..." required className="flex-1 px-
  4 py-2 border border-gray-300 rounded focus:ring-2 focus:ring-blue-500" />
  <button type="submit"
    className="px-4
    py-2 bg-blue-600
    text-white rounded
    hover:bg-blue-700
    disabled:opacity-50"
    disabled
    {!document.activeEl
    ement?.value?.trim()
    }
  >
    Add Task
  </button>
</form>
{/* Task list */}
<ul className="divide-y divide-gray-200">
  {tasks.map(task => (
    <li
      key={task.id} className="flex justify-between itemscenter
      px-4 py-4"
    >
      <span className={task.completed ? 'line-through text-gray-500' : ''}> {task.title}
      </span>
```

```
          <button onClick={() => handleDeleteTask(task.id)} className="px-3 py-1 bg-red-600 text-white
          textsm rounded hover:bg-red-700 disabled:opacity-50" disabled={task.isOptimistic} // Prevent clicking
          during optimistic state >
            Delete
          </button>
        </li>
      ))}
    </ul>
    {/* Empty state */}
    {tasks.length === 0 && (
      <p className="text-center py-8 text-gray-500">No tasks yet. Add one above!</p>
    )}
  </div>
 );
}
```

(b) Axios vs. Fetch Comparison

The code above uses axios for all API calls. Here's how it differs from the native fetch() used in Q3 (comment added directly in the code for clarity):

```
// AXIOS VS FETCH DIFFERENCES (NOTE FOR Q4b):
// 1. JSON Parsing:
//    - Axios: Automatically parses JSON response → `response.data`
//    - Fetch: Requires manual parsing → `await response.json()`
//
// 2. Error Handling:
//    - Axios: Rejects promise on 4xx/5xx status by default (can customize with validateStatus)
//    - Fetch: Only rejects on network errors; HTTP errors (404/500) resolve normally (must check response.ok)
//
// 3. Request Configuration:
```

// - Axios: Easier to set default headers/baseURL via instance (see `api = axios.create(...)`) // -

Fetch: Requires repeating headers/URL in every call

//

// 4. Body Stringification:

// - Axios: Automatically stringifies JSON objects when Content-Type is application/json // -

Fetch: Requires manual `JSON.stringify(body)`

//

// Example comparison for POST request:

// Axios (used above):

// await api.post('/tasks', {title}, {headers: {...}})

//

// Fetch equivalent (for reference):

// await fetch(`${import.meta.env.VITE_API_URL}/tasks`, {
// method: 'POST',

// headers: {

// 'Content-Type': 'application/json',

// 'x-api-key': import.meta.env.VITE_API_KEY

// },

// body: JSON.stringify({title})
// })

(c) Form & Delete Functionality The implemented

solution includes:

Task Creation Form:

Input field with validation (prevents empty tasks)

Optimistic UI update: Task appears immediately in list with temporary ID

On success: Replaces optimistic task with real server response (including correct id)

On error: Rolls back optimistic update and shows user-friendly error message

Form resets after submission **Delete**

Button Per Task:

Appears next to each task

LABSHEET 6 : FULL STACK DEVELOPMENT

Optimistic UI update: Task removed from list immediately

On success: Change permanently committed

On error: Task restored to list and error shown

Button disabled during optimistic state to prevent rapid clicks

Q5. Full-Stack Error Handling, Config & Final Integration

- **On the client, make sure a 404 or 401 response from the server shows a proper, readable error message in the UI (not a crash or blank screen). Test this by deleting a task using a fake id (e.g. 9999) from the React UI and confirm both server and client respond gracefully.**

- **Move every hardcoded value into config: PORT and the API key into the server's .env (confirmed in .gitignore), and the API's base URL into a React environment variable (e.g. VITE_API_URL) instead of a hardcoded localhost address.**

- **Combine everything into one working "Task Manager Mini App" — the Express API (with middleware, CORS, and error handling) plus the React frontend (with create/delete and loading/error states) — and submit both folders zipped together with your Postman collection export. ANS : (a) Client-Side Error Handling for 404/401 Responses**

Implementation (already in place from Q4b React client):

In client-react/src/App.jsx, the handleDeleteTask function includes: catch

```
(err) {
  // Rollback optimistic update setTasks(prev => [...prev, tasks.find(t => t.id === id)]); setError(
  err.response?.data?.error || // Extracts server's error message (e.g., "Task with id 9999 not found")
  err.message ||
   'Failed to delete task'
  );
}
```

The UI renders errors via:

```
if (error) return (
  <div className="p-4 bg-red-50 border border-red-200 text-red-800 rounded">
    <strong>Error:</strong> {error}
  </div>
);
```

Test Procedure for Fake ID Deletion (9999):

LABSHEET 6 : FULL STACK DEVELOPMENT

Start both servers: #

Terminal 1: API server

cd api-lab/server npm

run dev

Terminal 2: React dev server

cd api-lab/client-react npm run

dev

Open React app at http://localhost:5173

Attempt to delete a task with ID 9999:

Method 1 (Direct API call via browser console for testing):

```
fetch('http://localhost:5000/api/tasks/9999',  {
  method: 'DELETE', headers: { 'x-api-key':
  'your_secret_key_here' } })
```

Method 2 (Simulate via React UI - requires temporary code change):

Temporarily modify handleDeleteTask to accept a test ID:

// Add this line inside handleDeleteTask for testing ONLY:

// const testId = 9999;

// await api.delete(`/tasks/${testId}`, { headers: {...} }); Click any

"Delete" button → triggers call to /api/tasks/9999

Expected Results:

Server Response:

HTTP/1.1 404 Not Found

Content-Type: application/json

{ "error": "Task with id 9999 not found" }

Client UI:

Shows red alert box:

Error: Task with id 9999 not found

(Not a crash, blank screen, or raw stack trace)

Behavior:

Task list remains unchanged (optimistic update rolled back on error)

No JavaScript errors in browser console Page remains fully

interactive

Verification Complete: Both server and client respond gracefully to 404 errors with user-friendly messages.

(b) Configuration Hardening

All hardcoded values moved to environment variables with proper safeguards:

Server Configuration (api-lab/server/):

.env (create this file):

PORT=5000

API_KEY=your_strong_random_key_here # e.g., from `openssl rand -hex 16`

Usage in server.js: const PORT = process.env.PORT || 5000; // Fallback to

5000 const API_KEY

= process.env.API_KEY; // Used in auth-check middleware **.gitignore (verify**

this exists and contains):

Environment variables

.env

Dependency directories node_modules/

Logs

*.log

OS artifacts

.DS_Store

React Configuration (api-lab/client-react/):

.env (Vite-specific - must prefix with VITE_):

VITE_API_URL=http://localhost:5000/api

VITE_API_KEY=your_strong_random_key_here # Must match server's API_KEY

Usage in src/App.jsx: const api = axios.create({ baseURL:

import.meta.env.VITE_API_URL, // No hardcoded localhost

}); // In

headers:

```
headers: {
  'x-api-key': import.meta.env.VITE_API_KEY
}
```

Critical Vite Note:

Only variables prefixed with VITE_ are exposed to client code

VITE_API_URL and VITE_API_KEY are safely injected at build time

No hardcoded API URLs remain in client source

Verification Complete:

Zero hardcoded localhost:5000 or API keys in source code

.env files excluded from version control via .gitignore

All configuration externalized and environment-specific

(c) Full-System Integration Verification End-to-End

Test Flow (Task Manager Mini App):

Start Services:

```
# In separate terminals:
cd api-lab/server && npm run dev # → Server on :5000 cd
api-lab/client-react && npm run dev # → React on :5173
```

Verify Core Functionality:

- **GET /api/tasks (Plain JS client at client/index.html):** Shows loading state → renders task list Works without auth header (as required)
- **POST /api/tasks (React client):**
  Form submission creates task
  Requires x-api-key header (401 if missing)
  Optimistic UI update + server sync
- **DELETE /api/tasks/:id (React client):** Delete button removes task immediately
  Requires x-api-key header (401 if missing) Graceful
  404 handling for invalid IDs (tested in 5a)

LABSHEET 6 : FULL STACK DEVELOPMENT

- **PUT /api/tasks/:id (via Postman - from Q2b):**

  Updates task completion/title Requires auth header